

ITAI 5153 Project 1: Planning and Search - Experiment Results

Alec Pike

Setup

```
In [1]: import algorithms
        from solver import Solver
        from time import time
```

```
In [2]: # Initial example solver
        slv = Solver(algorithms.bfs)
        slv.cube.print_cube()
        print("Solved:", slv.is_solved())
```

Current cube state: WWWWGRRRBBBOOGRRRBBBOYYYYY

```
      W W
      W W
  G G  R R  B B  O O
  G G  R R  B B  O O
      Y Y
      Y Y
```

Solved: True

```
In [3]: # Example randomization
        slv.scramble(depth=5, rand_seed=42)
```

Current cube state: WBWGBBOWORYRGRGYBRYOWOYG

```
      W B
      W G
  B B  O W  O R  Y R
  G R  G Y  B R  Y O
      W O
      Y G
```

```
Out[3]: ((0, 3, 0, 1, 3, 3, 4, 0, 4, 2, 5, 2, 1, 2, 1, 5, 3, 2, 5, 4, 0, 4, 5, 1),
        ['u', 'F', 'F', 'u', 'R'])
```

```
In [4]: slv.is_solved()
```

Out[4]: False

Experiment

```
In [5]: def test_solver(scramble_depth: int):
        print("=" * 50)
        print()
        print("Testing with scramble depth =", scramble_depth)

        bfs_slv = Solver(algorithms.bfs)
        scramble_state, scramble_moves = bfs_slv.scramble(depth=scramble_depth, rand_seed=42)
        print("Scramble moves:", scramble_moves)

        print("-" * 50)
```

```

print("Solving with BFS...")
start_time = time()
bfs_solution, bfs_nodes = bfs_slv.solve()
bfs_time = time() - start_time
print("BFS solution:", bfs_solution)
print("Cost:", len(bfs_solution))
print("Nodes expanded:", bfs_nodes)
print("Time:", bfs_time)

print("-" * 50)
print("Solving with DFS (depth limit 15)...")
dfs_slv = Solver(algorithms.dfs, initial_state=scramble_state)
start_time = time()
dfs_solution, dfs_nodes = dfs_slv.solve(max_depth=15)
dfs_time = time() - start_time
print("DFS solution:", dfs_solution)
print("Cost:", len(dfs_solution))
print("Nodes expanded:", dfs_nodes)
print("Time:", dfs_time)

print("-" * 50)
print("Solving with A*...")
a_star_slv = Solver(algorithms.a_star, initial_state=scramble_state)
start_time = time()
a_star_solution, a_star_nodes = a_star_slv.solve()
a_star_time = time() - start_time
print("A* solution:", a_star_solution)
print("Cost:", len(a_star_solution))
print("Nodes expanded:", a_star_nodes)
print("Time:", a_star_time)

print()

```

In [6]: depths = [1, 3, 5, 10, 15, 25, 100]

```

for depth in depths:
    test_solver(depth)

```

=====

Testing with scramble depth = 1

Current cube state: WWWOOGGRRBBGGRRBBOOYYYY

```
W W
W W
O O G G R R B B
G G R R B B O O
Y Y
Y Y
```

Scramble moves: ['u']

Solving with BFS...

BFS solution: ['U']

Cost: 1

Nodes expanded: 1

Time: 2.2649765014648438e-05

Solving with DFS (depth limit 15)...

DFS solution: ['F', 'F', 'F', 'R', 'F', 'F', 'F', 'U', 'F', 'u', 'r', 'F', 'r', 'U', 'F']

Cost: 15

Nodes expanded: 112161

Time: 0.11492633819580078

Solving with A*...

A* solution: ['U']

Cost: 1

Nodes expanded: 2

Time: 0.11492633819580078

=====

Testing with scramble depth = 3

Current cube state: WWYYOBRGRBBGGRGOBOWWYY

```
W W
Y Y
O B R R G R B B
G R G G O B O O
W W
Y Y
```

Scramble moves: ['u', 'F', 'F']

Solving with BFS...

BFS solution: ['F', 'F', 'U']

Cost: 3

Nodes expanded: 8

Time: 4.506111145019531e-05

Solving with DFS (depth limit 15)...

DFS solution: ['F', 'F', 'F', 'R', 'F', 'F', 'F', 'R', 'f', 'r', 'U', 'R', 'u', 'F', 'r']

Cost: 15

Nodes expanded: 68908

Time: 0.07204127311706543

Solving with A*...

A* solution: ['F', 'F', 'U']

Cost: 3

Nodes expanded: 7

Time: 0.07204127311706543

=====

Testing with scramble depth = 5

Current cube state: WBWGBBOWORYRGRGYBRYOWOYG

```
W B
W G
B B O W O R Y R
G R G Y B R Y O
W O
Y G
```

Scramble moves: ['u', 'F', 'F', 'u', 'R']

Solving with BFS...

BFS solution: ['r', 'U', 'F', 'F', 'U']

Cost: 5

Nodes expanded: 551

Time: 0.019077777862548828

Solving with DFS (depth limit 15)...

DFS solution: ['F', 'F', 'F', 'R', 'R', 'R', 'F', 'U', 'f', 'U', 'f', 'u', 'R', 'f', 'U']

Cost: 15

Nodes expanded: 14165569

Time: 16.315587282180786

Solving with A*...

A* solution: ['r', 'U', 'F', 'F', 'U']

Cost: 5

Nodes expanded: 73

Time: 16.315587282180786

=====

Testing with scramble depth = 10

Current cube state: GRORRBYWGBWYGBYWOWOROYG

```
G R
O R
R B Y W G B W Y
G B Y B W O W O
R O
Y G
```

Scramble moves: ['u', 'F', 'F', 'u', 'R', 'R', 'R', 'u', 'F', 'u']

Solving with BFS...

BFS solution: ['F', 'F', 'r', 'F', 'u', 'f']

Cost: 6

Nodes expanded: 737

Time: 0.0042231082916259766

Solving with DFS (depth limit 15)...

DFS solution: ['F', 'F', 'F', 'R', 'F', 'F', 'R', 'U', 'R', 'F', 'F', 'F', 'R', 'F']

Cost: 14

Nodes expanded: 583142

Time: 0.6496953964233398

Solving with A*...

A* solution: ['f', 'f', 'r', 'F', 'u', 'f']

Cost: 6

Nodes expanded: 318

Time: 0.6496953964233398

=====

Testing with scramble depth = 15

Current cube state: ROOGWGWYRWBBGBYOYGWORBYR

```

      R O
      O G
W G  W Y  R W  B B
G B  Y O  Y G  W O
      R B
      Y R

```

Scramble moves: ['u', 'F', 'F', 'u', 'R', 'R', 'R', 'u', 'F', 'u', 'u', 'r', 'F', 'r', 'f']

Solving with BFS...

BFS solution: ['F', 'R', 'f', 'R', 'U', 'F', 'F', 'r', 'F', 'u', 'f']

Cost: 11

Nodes expanded: 628695

Time: 7.279831886291504

Solving with DFS (depth limit 15)...

DFS solution: ['F', 'F', 'F', 'U', 'R', 'F', 'R', 'u', 'f', 'U', 'U', 'F', 'u', 'r', 'F']

Cost: 15

Nodes expanded: 70004192

Time: 78.77350878715515

Solving with A*...

A* solution: ['u', 'F', 'r', 'u', 'r', 'F', 'u', 'F', 'u', 'r', 'U']

Cost: 11

Nodes expanded: 260818

Time: 78.77350878715515

=====

Testing with scramble depth = 25

Current cube state: BYBBYWROWRGRGOYGORGOWBYW

```

      B Y
      B B
Y W  R O  W R  G R
G O  Y G  O R  G O
      B W
      Y W

```

Scramble moves: ['u', 'F', 'F', 'u', 'R', 'R', 'R', 'u', 'F', 'u', 'u', 'r', 'F', 'r', 'f', 'R', 'R', 'F', 'r', 'u', 'u', 'u', 'r', 'f', 'R']

Solving with BFS...

BFS solution: ['F', 'F', 'r', 'U', 'R', 'R', 'f', 'r', 'F', 'r', 'f']

Cost: 11

Nodes expanded: 563179

Time: 5.930660963058472

Solving with DFS (depth limit 15)...

DFS solution: ['F', 'F', 'F', 'R', 'R', 'R', 'f', 'U', 'r', 'U', 'r', 'f', 'R', 'R', 'f']

Cost: 15

Nodes expanded: 15032703

Time: 18.014995098114014

Solving with A*...

A* solution: ['U', 'R', 'u', 'F', 'r', 'f', 'U', 'f', 'r', 'F', 'F']

Cost: 11

Nodes expanded: 261832

Time: 18.014995098114014

=====

Testing with scramble depth = 100

Current cube state: OWRRYGYWGRBBGBWRBG000YYW

O W

```

      R R
    Y G Y W G R B B
    G B W R B G O O
      O Y
      Y W

```

Scramble moves: ['u', 'F', 'F', 'u', 'R', 'R', 'R', 'u', 'F', 'u', 'u', 'r', 'F', 'r', 'f', 'R', 'R', 'F', 'r', 'u', 'u', 'u', 'r', 'f', 'R', 'f', 'r', 'U', 'F', 'R', 'u', 'f', 'U', 'U', 'R', 'R', 'U', 'F', 'F', 'F', 'U', 'U', 'r', 'U', 'F', 'u', 'f', 'r', 'F', 'F', 'r', 'U', 'r', 'U', 'r', 'u', 'F', 'F', 'u', 'R', 'U', 'F', 'R', 'F', 'U', 'f', 'u', 'R', 'U', 'U', 'R', 'u', 'u', 'u', 'u', 'F', 'r', 'u', 'R', 'u', 'R', 'R', 'f', 'f', 'U', 'r', 'u', 'F', 'R', 'F', 'U', 'f', 'U', 'F', 'R', 'u', 'R', 'u', 'f', 'f']

Solving with BFS...

BFS solution: ['R', 'f', 'R', 'U', 'f', 'R', 'F', 'U', 'U', 'F']

Cost: 10

Nodes expanded: 303549

Time: 3.5207536220550537

Solving with DFS (depth limit 15)...

DFS solution: ['F', 'F', 'F', 'R', 'F', 'R', 'f', 'R', 'U', 'f', 'R', 'F', 'U', 'F']

Cost: 14

Nodes expanded: 3698029

Time: 3.9927282333374023

Solving with A*...

A* solution: ['R', 'f', 'R', 'U', 'f', 'R', 'F', 'U', 'U', 'F']

Cost: 10

Nodes expanded: 77636

Time: 3.9927282333374023

```

In [7]: # Just for fun, let's also run DFS without a depth limit
dfs_no_limit = Solver(algorithms.dfs)
scramble_state, scramble_moves = dfs_no_limit.scramble(depth=15, rand_seed=42)
print("Scramble moves:", scramble_moves)
print()
print("Running DFS with no depth limit...")
start_time = time()
dfs_no_limit_solution, dfs_no_limit_nodes = dfs_no_limit.solve()
dfs_no_limit_time = time() - start_time
print("DFS no limit solution:", dfs_no_limit_solution)
print("Cost:", len(dfs_no_limit_solution))
print("Nodes expanded:", dfs_no_limit_nodes)
print("Time:", dfs_no_limit_time)

```

Current cube state: ROOGWGWYRWBBGBYOYGWORBYR

```

      R O
      O G
    W G W Y R W B B
    G B Y O Y G W O
      R B
      Y R

```

Scramble moves: ['u', 'F', 'F', 'u', 'R', 'R', 'R', 'u', 'F', 'u', 'u', 'r', 'F', 'r', 'f']

Running DFS with no depth limit...

RecursionError Traceback (most recent call last)

Cell In[7], line 8

```
4 print("Scramble moves:", scramble_moves)
5 print()
6 print("Running DFS with no depth limit...")
7 start_time = time()
----> 8 dfs_no_limit_solution, dfs_no_limit_nodes = dfs_no_limit.solve()
9 dfs_no_limit_time = time() - start_time
10 print("DFS no limit solution:", dfs_no_limit_solution)
11 print("Cost:", len(dfs_no_limit_solution))
```

File /mnt/c/Users/apike/Desktop/GitHub repos/2x2Rubk-sCubeSolver/solver.py:84, in Solver.solve(self, apply_moves, **kwargs)

```
81     return [], 0
83 # Pass state and successor generator to the algorithm
----> 84 solution_path, nodes_explored = self.alg(
85     start_state=self.cube.state,
86     goal_state=PocketCube.DEFAULT_STATE,
87     get_successors=Solver.get_successors,
88     **kwargs
89 )
91 if solution_path and apply_moves:
92     for move in solution_path:
```

File /mnt/c/Users/apike/Desktop/GitHub repos/2x2Rubk-sCubeSolver/algorithms.py:80, in dfs(start_state, goal_state, get_successors, max_depth)

```
76         path_visited.remove(next_state)
78     return None
----> 80 return _dfs(start_state, [], nodes_explored)
```

File /mnt/c/Users/apike/Desktop/GitHub repos/2x2Rubk-sCubeSolver/algorithms.py:71, in dfs.<locals>._dfs(current_state, path)

```
68 if next_state not in path_visited:
69     path_visited.add(next_state)
----> 71 result = _dfs(next_state, path + [move])
72 if result is not None:
73     return result
```

File /mnt/c/Users/apike/Desktop/GitHub repos/2x2Rubk-sCubeSolver/algorithms.py:71, in dfs.<locals>._dfs(current_state, path)

```
68 if next_state not in path_visited:
69     path_visited.add(next_state)
----> 71 result = _dfs(next_state, path + [move])
72 if result is not None:
73     return result
```

[... skipping similar frames: dfs.<locals>._dfs at line 71 (970 times)]

File /mnt/c/Users/apike/Desktop/GitHub repos/2x2Rubk-sCubeSolver/algorithms.py:71, in dfs.<locals>._dfs(current_state, path)

```
68 if next_state not in path_visited:
69     path_visited.add(next_state)
----> 71 result = _dfs(next_state, path + [move])
72 if result is not None:
73     return result
```

File /mnt/c/Users/apike/Desktop/GitHub repos/2x2Rubk-sCubeSolver/algorithms.py:66, in dfs.<locals>._dfs(current_state, path)

```
63 if max_depth is not None and len(path) >= max_depth:
64     return None
```

```
---> 66 for next_state, move in get_successors(current_state):
      67     # Avoid cycles along the currentbranch path
      68     if next_state not in path_visited:
      69         path_visited.add(next_state)
```

File /mnt/c/Users/apike/Desktop/GitHub repos/2x2Rubk-sCubeSolver/solver.py:63, in Solver.get_successors(state)

```
      61 for move in all_moves:
      62     temp_cube = PocketCube(state)
---> 63     temp_cube.make_move(move)
      64     successors.append((temp_cube.state, move))
      65 return successors
```

File /mnt/c/Users/apike/Desktop/GitHub repos/2x2Rubk-sCubeSolver/pocketcube.py:94, in PocketCube.make_move(self, move)

```
      92     return True
      93 case "F":
---> 94     self._turn_F()
      95     return True
      96 case "G" | "f":
```

RecursionError: maximum recursion depth exceeded

In []: